

American University of Phnom Penh



Final Project – Photo Booth

ICT 360 – Introduction to Internet of Things

Prof. Theara SENG

Group 9

ROTH Noreasakpheak

CHHOY Sokunthanou

CHHOM Alead

6 May, 2026

I. Problem Statement

Traditional professional photobooths, while popular at events, present several significant hurdles for casual or DIY deployment:

- **High Cost:** professional units are expensive to rent, and building one typically requires high-end hardware like DSLRs and dedicated PCs
- **Large Physical Footprint:** most systems are bulky, heavy, and lack portability, making them difficult to transport to different venues
- **Manual Sharing Processes:** many digital cameras still rely on SD card transfers or cables, preventing instant social media sharing
- **Lack of Remote Interaction:** physical interaction with the booth hardware is usually required, with no native wireless trigger mechanisms available for users

II. Proposed Solution

To address these issues, the IoT Photobooth System utilizes a dual-microcontroller architecture to provide a modern, affordable, and wireless experience:

- **Wireless Control:** users interact with the booth via any mobile browser, eliminating the need for dedicated app installations
- **Affordability:** built using low-cost, off-the-shelf IoT components like the ESP32 and ESP32-CAM
- **Instant Delivery:** captured photos are pushed directly to a Telegram group using HTTPS for immediate digital access
- **User Feedback:** integrated LCDs, RGB LED rings, and audio buzzers provide real-time guidance and countdowns

III. System Overview

The system is designed to be a "palm-sized" photobooth that automates the photography process from proximity detection to cloud delivery.

Key Features:

- **Automated Guidance:** an ultrasonic sensor monitors user distance to ensure they are in the "perfect shot" position
- **Multi-Sensory Countdown:** synchronized visual and audio cues (LED ring, LCD, and Buzzer) notify the user exactly when the capture occurs
- **Dual-Core Processing:** manages simultaneous video streaming and image capturing without system crashes

IV. Hardware Components

1. **ESP32:** acts as the central controller; manages the Web Server, user proximity (HC-SR04), LCD display, and the NeoPixel LED ring
2. **ESP32-CAM:** dedicated camera server; handles the MJPEG livestream, image sensor resolution, and HTTPS uploads to Telegram
3. **Ultrasonic sensor:** used to monitor user distance and trigger "Ready" states
4. **NeoPixel Ring:** provides visual countdown feedback through color transitions
5. **I2C LCD:** displays status/IP information
6. **Buzzer:** provides audible countdown beeps and alert when people stand too close to the camera (ESP32-CAM)

1. ESP32



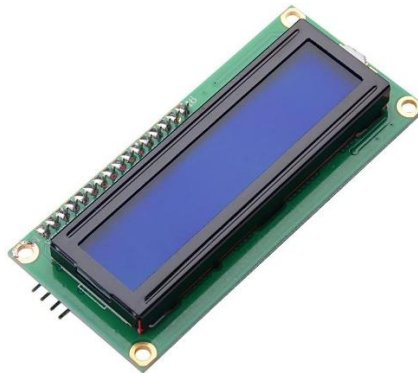
2. ESP32-CAM



3. Ultrasonic sensor



4. NeoPixel Ring



5. I2C LCD



6. Buzzer

V. Setup Guide – Arduino IDE

Package

- ESP32CAM_Photobooth/ESP32CAM_Photobooth.ino (Flash to ESP32-CAM)
- MainESP32_Photobooth/MainESP32_Photobooth.ino (Flash to Main ESP32)

Wiring References

Main ESP32 DevKit v1

- **LCD I2C 16x2**

LCD Pin	ESP32 Pin
VCC	3.3V
GND	GND
SDA	GPIO21
SCL	GPIO22

- **WS2812B LED Ring**

LED Pin	ESP32 Pin
5V	5V (external or from board)
GND	GND
DIN	GPIO5 (<i>put a 330Ω resistor on this wire!</i>)

- **Active Buzzer**

Buzzer Pin	ESP32 Pin
+	GPIO18
GND	GND

- **HC-SR04 Ultrasonic Sensor**

ECHO outputs 5V — use a 1k Ω /2k Ω voltage divider to bring it to ~3.3V for the ESP32 input

Sensor Pin	ESP32 Pin
VCC	5V
GND	GND
TRIG	GPIO27
ECHO	GPIO14

ESP32-CAM

FTDI Adapter (for uploading firmware)

FTDI	ESP32-CAM
5V	5V
GND	GND
TX	U0R (GPIO3)
RX	U0T (GPIO1)
—	GPIO0 $\rightarrow$ GND <i>only during upload, remove after!</i>

Normal Operation

- Power via 5V 2A supply (*keep separate from Main ESP32*)
- Flash LED is onboard (GPIO4), no extra wiring needed

Setup & Config:

Step 1: Download and install Arduino IDE: <https://www.arduino.cc/en/software>

Step 2: Add ESP32 Board Package:

1. Open Arduino IDE → **File** → **Preferences**

2. In "**Additional Boards Manager URLs**" paste:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

3. Click **OK**

4. Go to **Tools** → **Board** → **Boards Manager**

5. Search esp32 → Install "**esp32 by Espressif Systems**" (*takes a few minutes*)

Step 3: Install Required Libraries

Go to **Tools** → **Manage Libraries**

Library	Author	Board
LiquidCrystal I2C	Frank de Brabander	Main ESP32
Adafruit NeoPixel	Adafruit	Main ESP32

Note: WiFi, WebServer, HTTPClient, Wire, and esp_camera are all bundled with the ESP32 Arduino core, no separate install needed.

Step 4: Fill in config values

ESP32CAM_Photobooth.ino – edit the top section:

```
WIFI_SSID   = "your network name"

WIFI_PASSWORD = "your password"

BOT_TOKEN   = "your Telegram bot token"

CHAT_ID     = "your Telegram group chat ID"    // Groups have a negative ID e.g. -1001234567890
```

MainESP32_Photobooth.ino — edit the top section:

```
WIFI_SSID  = "your network name"

WIFI_PASSWORD = "your password"

ESP32CAM_IP  = // leave as-is for now, set after Step 6

NUM_LEDS    = // change to match your actual LED ring count
```

Steps 5: Upload to ESP32-CAM (do this first)

Upload Steps:

1. **Tools** → **Board** → "AI Thinker ESP32-CAM"
2. **Tools** → **Port** → select your FTDI COM port
3. **Tools** → **Upload Speed** → 115200 (*try lower if upload fails*)
4. Open ESP32CAM_Photobooth.ino and click **Upload**
5. When you see Connecting....., in the output, press and hold **RESET** on the ESP32-CAM, then release, upload should start
6. Wait for "**Done uploading**"
7. **Remove** the GPIO0 → GND wire
8. Open **Serial Monitor** (Tools → Serial Monitor), set **115200 baud**
9. Press **RESET** on the ESP32-CAM
10. Watch for the IP address e.g. ESP32-CAM IP: 192.168.1.101
11. **Copy that IP** — you need it in the next step

Step 6: Set the Cam IP in Main ESP32 Sketch

1. Open MainESP32_Photobooth.ino
2. Find

```
const char* ESP32CAM_IP = "192.168.x.x";
```

3. Replace with the IP you just copied, e.g. "192.168.1.101"
4. Save the file

Step 7: Upload to Main ESP32

1. Connect Main ESP32 DevKit to PC via USB
2. In Arduino IDE:
 - **Tools** → **Board** → "ESP32 Dev Module"
 - **Tools** → **Port** → select the Main ESP32 COM port
3. Open MainESP32_Photobooth.ino and click **Upload**
4. Open **Serial Monitor** at **115200 baud**
5. Reset the board — watch for "Server ready: http://..."
6. The LCD will display the IP address

Step 8: Every Time You Use It

1. Power on **ESP32-CAM** first
2. Power on **Main ESP32**
3. Wait for LCD to show the IP address
4. On your phone — connect to the **same WiFi network**
5. Open browser → type the IP shown on LCD (e.g. http://192.168.1.100)
6. Tap "**3 sec**" or "**5 sec**" to start a countdown
7. Photo is taken and sent to Telegram!

VI. System Architecture & Workflow

System Architecture

controller, coordinates hardware, and handles web serve an user interaction while the ESP32-CAM manages data-heavy media tasks, such as image capturing, MJPEG livestream, and Telegram delivery.



System Workflow:

1. **Connection:** both boards boot and connect to the same local Wi-Fi network, the Main ESP32 displays its IP address on the LCD once connected
2. **Access:** the user enters the IP in a browser to see a live MJPEG stream, resolution setting and capture controls
3. **Trigger:** when "Start" is tapped, the browser sends a request to the Main ESP32, which triggers the ESP32-CAM following the triggered action
4. **Countdown:** the web displays a visual countdown, the LCD, LED ring, and Buzzer executes a synchronized countdown
5. **Capture:** At zero, the ESP32-CAM pauses the stream, fires the flash, and captures a JPEG
6. **Delivery:** the image is uploaded via HTTPS to the Telegram Bot API, the result is shown on the LCD, the NeoPixel shows green for success and red for failure

VII. Technical Implementation

- **Dual-Port Web Server:** the ESP32-CAM uses Port 80 for commands and Port 81 for independent MJPEG streaming
- **FreeRTOS Multitasking:** tasks are distributed across cores to ensure capture requests never block the live video feed
- **Mutex Protection:** Mutex-protected camera buffers prevent frame conflicts during simultaneous operations
- **Self-contained web interface:** the HTML control interface is stored directly in the sketch, requiring no external SD cards for the web files

VIII. System Limitations

- **Connectivity:** both the user and the devices must remain on the same local Wi-Fi network, offline mode is not supported
- **Configuration:** the ESP32-CAM IP address currently requires manual updates
- **Functionality:** supports only one photo per trigger with a fixed resolution and lacks user authentication
- **Power:** requires a stable 5V 2A power supply to handle the current spikes of the camera and flash

IX. Future Improvements

- **mDNS Support:** implement mDNS for automatic IP discovery
- **QR Integration:** display a dynamic QR code on the LCD for instant browser access
- **Physical Backup:** add a hardware shutter button as a failsafe trigger
- **Cloud Expansion:** support additional destinations like Google Drive, Firebase, or thermal printers